



FOR JAVA DEVELOPERS MOVING TO NESTJS

One slow client. 48 MB.

Spring Boot to NestJS, episode 30: WebSockets

A realtime connection is not just a socket. Choose the protocol, how messages cross instances, and what happens when a client cannot keep up.

A client that stops reading: Spring closed the session at its 512 KB send buffer limit. NestJS kept it open, with about 48 MB queued for that one socket.

A plain WebSocket client could not connect to Nest's default gateway, which speaks Socket.IO. With WsAdapter it could.

Two instances, one client on each: the defaults on both stacks reached one of two. A shared broker reached both.

In scope

- @MessageMapping and @SubscribeMessage
- STOMP, Socket.IO and plain WebSocket
- Fan out across instances
- Slow clients

Not in scope

- Authentication on the handshake
- Server sent events (next episode)

At the time of recording Nest's default WebSocket adapter is Socket.IO.

01 The measurements

PLAIN CLIENT, NEST DEFAULT GATEWAY	failed: socket hang up
PLAIN CLIENT, NEST WsAdapter	connected, broadcast received
TWO INSTANCES, DEFAULTS, BOTH STACKS	instance two received 0
RABBITMQ RELAY / REDIS ADAPTER	both received it
SLOW CLIENT, SPRING DEFAULTS	closed at the 512 KB limit
SLOW CLIENT, NEST	about 48 MB queued after 15 s
NEST WITH A 512 KB CHECK	disconnected; bounded

Every server a real process; a slow client is one whose TCP socket stopped reading.

02 Bound the queue per client

Before every send, check how much is already queued for that client. Past your limit, disconnect it (messages intact, connection lost) or drop and merge updates (connection kept, messages lost). Choose one on purpose.

payments.gateway.ts

```
const LIMIT = 512 * 1024;
function send(client: WebSocket, data: string) {
  if (client.readyState !== client.OPEN) return;
  if (client.bufferedAmount > LIMIT) {
    client.terminate();
    return;
  }
  client.send(data);
}
```

with the 512 KB check: terminated once the queue crossed 524288 bytes

Run on a real machine, 2026-10-01, by scripts/verify-websockets.sh in the course repository.

03 Fan out across instances

A WebSocket connection lives on one process. Spring needs a broker relay; Nest's Socket.IO needs an adapter such as Redis.

main.ts

```
const pub = new Redis(url);
const sub = pub.duplicate();
server.adapter(createAdapter(pub, sub));
```

WebSocketConfig.java

```
registry.enableStompBrokerRelay("/topic").setRelayPort(relayPort);
```

two instances: RabbitMQ relay 1 and 1, Redis adapter 1 and 1

Run on a real machine, 2026-10-01, by scripts/verify-websockets.sh in the course repository.

Before you ship a WebSocket gateway

1. What protocol do your clients speak: Socket.IO, STOMP or plain WebSocket?
2. Does a broadcast reach clients on every instance?
3. What happens when one client stops reading?
4. Disconnect or drop: which promise does each message type need?
5. How does a client catch up after a reconnect?

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 X https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-websockets.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS gateways https://docs.nestjs.com/websockets/gateways

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. RabbitMQ is a trademark of Broadcom. Redis is a trademark of Redis Ltd. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.